

Roll Number:-2070120

Name:-Samiksha Sanjay Adake

Assignment Number:-1

Problem Statement:-Write a program to implement a class for Book details as follows Private: bookid, book_name, author, price Public: get_details(), print_details() Get details of some books from user and print in tabular form. Also find the total prize of all the books.

Theory:-

1. Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm in which a program is designed using classes and objects. It combines data (variables) and functions (methods) into a single unit called a class. OOP makes programs more organized, secure, reusable, and easier to maintain. The four main principles of OOP are Encapsulation, Inheritance, Polymorphism, and Abstraction. In this program, OOP is implemented using the Book class, where book details and the functions that operate on them are grouped together.

2. Class

A class is a user-defined data type in C++ that acts as a blueprint or template for creating objects. It combines data members (variables) and member functions (methods) into a single unit. A class helps organize data and functions

together, making programs easier to understand and maintain. In this program, Book is a class that stores the details of a book, such as its ID, name, author, and price, along with functions to input and display these details.

3. Object

An object is an instance of a class. It is used to access the data members and member functions of the class. Every object has its own copy of the data members. In this program, `Book b[20];` creates an array of 20 objects of the Book class, allowing the program to store information for up to 20 different books.

4. Data Members

Data members are the variables declared inside a class. They represent the properties or attributes of an object. In this program, `bookid`, `book_name`, `author`, and `price` are data members because they store the details of each book.

5. Member Functions

Member functions are functions defined inside a class that perform operations on the data members. They are used to input, process, or display the data stored in an object. In this program, `get_details()`, `print_details()`, and `getPrice()` are member functions of the Book class.

6. Access Specifiers

Access specifiers control the accessibility of the members of a class. The most common access specifiers are `private`, `public`, and `protected`. In this program, the data members are declared under `private`, so they cannot be accessed directly from outside the class. The member functions are declared

under public, allowing them to be called from the main() function.

7. Private Access Specifier

The private access specifier is used to hide the data members from direct access outside the class. Only the member functions of the same class can access private members. In this program, bookid, book_name, author, and price are private, which protects the data from unauthorized modification.

8. Public Access Specifier

The public access specifier allows the member functions and variables to be accessed from anywhere in the program. In this program, the functions get_details(), print_details(), and getPrice() are public, so they can be called using the object b[i] in the main() function.

9. Encapsulation

Encapsulation is the process of combining data members and member functions into a single unit called a class. It also protects the data by restricting direct access using access specifiers. In this program, the Book class encapsulates the book details and the functions that operate on those details, providing better security and organization.

10. Data Hiding

Data hiding is an important feature of Object-Oriented Programming that protects the data from direct access by making it private. Users cannot change the values of private variables directly; instead, they must use public member functions. In this program, the book details are hidden inside

the class and are accessed only through the provided functions.

11. Function

A function is a block of code that performs a specific task. It helps reduce code repetition and makes programs easier to understand. In this program, `get_details()` accepts book information, `print_details()` displays the book details, and `getPrice()` returns the price of the book.

12. Inline Function

An inline function is a function in which the compiler attempts to replace the function call with the actual function code, reducing function call overhead. In C++, a function defined inside a class is automatically treated as an inline function unless specified otherwise. In this program, the function `getPrice()` is an inline function because it is defined inside the class and simply returns the value of price.

13. Array of Objects

An array of objects is a collection of objects of the same class stored in consecutive memory locations. It is useful when multiple objects of the same type are required. In this program, `Book b[20];` creates an array that can store the details of 20 different books.

14. Loop

A loop is a programming statement that repeats a block of code until a specified condition becomes false. In this program, the for loop is used to enter the details of all books and again to display the details and calculate the total price.

15. Function Calling

Function calling means executing a function whenever its task is needed. A function is called using the object and the dot (.) operator. In this program, `b[i].get_details();` calls the function to accept book details, while `b[i].print_details();` displays the stored information.

16. Dot (.) Operator

The dot (.) operator is used to access the public members of an object. It connects the object with its member function or variable. In this program, `b[i].get_details()` and `b[i].print_details()` use the dot operator to call the member functions of the Book object.

17. Return Function

A return function is a function that sends a value back to the calling function using the return statement. In this program, the `getPrice()` function returns the value of the price data member, which is then added to calculate the total price of all books.

18. Void Function

A void function is a function that does not return any value. It performs a task but does not send any result back to the caller. In this program, `get_details()` and `print_details()` are void functions because they only take input and display output.

19. Main Function

The `main()` function is the starting point of every C++ program. Program execution begins from this function. In this program, `main()` creates the objects, accepts the number of

books, calls the member functions, displays the details, and calculates the total price.

20. Input and Output

The input and output statements allow the user to interact with the program. The cin object is used to take input from the keyboard, while the cout object is used to display information on the screen. In this program, cin is used to enter book details, and cout is used to display the entered information.

21. setw() Manipulator

The setw() manipulator is defined in the <iomanip.h> header file and is used to set the width of the next output field. It helps display data in a tabular and properly aligned format. In this program, setw() is used to align the Book ID, Book Name, Author, and Price into neat columns, making the output easier to read.

Code:

```
#include <iostream.h>
#include<conio.h>
#include <iomanip.h>

class book
{
private:
    int bookid, price;
    char book_name[50], author[50];
```

```
public:
    void get_details()
    {
        cout << "Enter Book ID: ";
        cin >> bookid;

        cout << "Enter Book Name: ";
        cin >> book_name;

        cout << "Enter Author Name: ";
        cin >> author;

        cout << "Enter Price: ";
        cin >> price;
    }

    void print_details()
    {
        cout << setw(10) << bookid
            << setw(20) << book_name
            << setw(20) << author
            << setw(10) << price << endl;
    }

    int get_price()
    {
        return price;
    }
```

```
};
```

```
int main()
```

```
{
```

```
    clrscr();
```

```
    int i, total = 0;
```

```
    book b[4];
```

```
    for(i = 0; i < 4; i++)
```

```
    {
```

```
        cout << "\nEnter details of Book " << i + 1 << endl;
```

```
        b[i].get_details();
```

```
    }
```

```
    cout << "\n";
```

```
    cout << setw(10) << "Book ID"
```

```
        << setw(20) << "Book Name"
```

```
        << setw(20) << "Author"
```

```
        << setw(10) << "Price" << endl;
```

```
    cout << "-----" <<
```

```
endl;
```



```
for(i = 0; i < 4; i++)
{
    b[i].print_details();
    total += b[i].get_price();
}

cout << "-----" <<
endl;
cout << "Total Price of All Books = " << total << endl;

return 0;
}
```

Output:

Book ID	Book Name	Author	Price
101	Clanuage	DennisRitchie	450
102	DBMS	Korth	550
103	OperatingSystem	Galvin	700
104	ComputerNetworks	Forouzan	600
Total Price of All Books = 2300			

Outcomes:

- Understand the concept of classes and objects in C++.
- Learn to use private data members and public member functions.
- Implement encapsulation for data security.
- Use an array of objects to store details of multiple books.
- Display data in a tabular format.
- Calculate the total price of all the books using arithmetic operations.